

บทที่ 10 : การพัฒนาเว็บ แอปพลิเคชันส่วนแบ็กเอนด์

1. เริ่มต้นสร้าง API
2. สร้าง API ตามหลักการของ CRUD
3. สร้าง Register API
4. สร้าง Login API
5. สร้าง Profile API
6. แบบฝึกหัด



1. เริ่มต้นสร้าง API

1. สร้างโฟลเดอร์โปรเจกต์

```
mkdir my-express-api  
cd my-express-api  
npm init -y
```

} ถ้ามี folder อยู่แล้วสามารถใช้ VS Code เปิดไปที่ folder ได้เลย

2. ติดตั้ง Express และ TypeScript

```
npm install express  
npm install -D typescript ts-node-dev @types/node @types/express
```

3. สร้างไฟล์ tsconfig.json

```
npx tsc --init
```

4. แก้ไขไฟล์ tsconfig.json

```
"compilerOptions": {  
  "rootDir": "./src", // Source files will be in 'src' folder  
  "outDir": "./dist", // Compiled JS will go to 'dist' folder  
}
```

5. แก้ไขไฟล์ package.json

```
"scripts": {  
  "dev": "ts-node-dev --respawn --transpile-only src/index.ts",  
  "build": "tsc",  
  "start": "node dist/index.js"  
}
```

6. เขียน API เบื้องต้น โดยสร้างไฟล์ src/index.ts ดังนี้

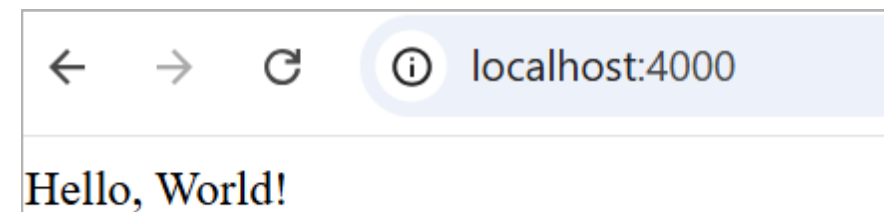
```
1  //1. Importing Dependencies
2  import express from 'express'; //นำเข้าโมดูล express
3
4  //2. Configuration
5  const app = express(); //กำหนดตัวแปร app เพื่อใช้สำหรับสร้าง API
6  const PORT = process.env.PORT || 4000; //กำหนด port การทำงานของ API server
7
8  //3. Defining API Endpoints
9  // Hello World
10 app.get('/', (req, res) => {
11   res.send('Hello, World!'); // ส่งข้อความ "Hello, World!" เมื่อเข้าถึง root path
12 });
13
14 //4. Starting Web Server
15 app.listen(PORT, () => {
16   console.log(`Server is running at http://localhost:${PORT}`);
17 });
```

7. รัน API

npm run dev

8. ทดสอบเรียก url จาก web browser หรือ Postman

http://localhost:4000



2. สร้าง API ตามหลักการของ CRUD

CRUD Operation	Example	HTTP Request Method	SQL command
Read	Show user profile, product detail, customer list, product list	GET	SELECT
Create	Add customer, add product, Register	POST	INSERT
Update	Update customer, update product	PUT	UPDATE
Delete	Delete customer, delete product	DELETE	DELETE

*Login ไม่ถือว่าเป็น CRUD Operation ซึ่งโดยปกติใช้ POST Method และ SELECT command

ใช้ MySQL Workbench สร้างฐานข้อมูล (Schema) ชื่อ shopdee แล้วสร้างตาราง customer ดังนี้

Column Name	Datatype	PK	NN	UQ	B	UN	ZF	AI	G	Default/Expression
 custID	INT	☒	☒	☐	☐	☒	☐	☒	☐	
 username	VARCHAR(20)	☐	☒	☐	☐	☐	☐	☐	☐	
 password	VARCHAR(100)	☐	☒	☐	☐	☐	☐	☐	☐	
 firstName	VARCHAR(100)	☐	☒	☐	☐	☐	☐	☐	☐	
 lastName	VARCHAR(100)	☐	☒	☐	☐	☐	☐	☐	☐	
 address	VARCHAR(200)	☐	☐	☐	☐	☐	☐	☐	☐	NULL
 mobilePhone	CHAR(10)	☐	☐	☐	☐	☐	☐	☐	☐	NULL
 birthdate	DATE	☐	☐	☐	☐	☐	☐	☐	☐	NULL
 gender	TINYINT	☐	☐	☐	☐	☒	☐	☐	☐	NULL
 email	VARCHAR(100)	☐	☐	☐	☐	☐	☐	☐	☐	NULL
 imageFile	VARCHAR(100)	☐	☐	☐	☐	☐	☐	☐	☐	NULL
 loginAttempt	TINYINT	☐	☐	☐	☐	☒	☐	☐	☐	'0'
 lastAttemptTime	DATETIME	☐	☐	☐	☐	☐	☐	☐	☐	NULL
 isActive	TINYINT	☐	☒	☐	☐	☒	☐	☐	☐	'1'

หรือจะใช้คำสั่ง SQL ดังนี้

```
CREATE TABLE `customer` (  
  `custID` int unsigned NOT NULL AUTO_INCREMENT COMMENT 'รหัสผู้ใช้งาน',  
  `username` varchar(20) NOT NULL COMMENT 'ชื่อผู้ใช้',  
  `password` varchar(100) NOT NULL COMMENT 'รหัสผ่าน',  
  `firstName` varchar(100) NOT NULL COMMENT 'ชื่อ',  
  `lastName` varchar(100) NOT NULL COMMENT 'นามสกุล',  
  `address` varchar(200) DEFAULT NULL COMMENT 'ที่อยู่',  
  `mobilePhone` char(10) DEFAULT NULL COMMENT 'หมายเลขโทรศัพท์เคลื่อนที่',  
  `birthdate` date DEFAULT NULL COMMENT 'วันเกิด',  
  `gender` tinyint unsigned DEFAULT NULL COMMENT 'เพศ (0=ชาย, 1=หญิง)',  
  `email` varchar(100) DEFAULT NULL COMMENT 'อีเมล',  
  `imageFile` varchar(100) DEFAULT NULL COMMENT 'ไฟล์รูปภาพ',  
  `loginAttempt` tinyint unsigned DEFAULT '0',  
  `lastAttemptTime` datetime DEFAULT NULL,  
  `isActive` tinyint unsigned NOT NULL DEFAULT '1' COMMENT 'สถานะการเป็นสมาชิก (0=ยกเลิกสมาชิก, 1=เป็นสมาชิก)',  
  PRIMARY KEY (`custID`)  
) ENGINE=InnoDB AUTO_INCREMENT=1 DEFAULT CHARSET=utf8mb3;
```

1. ติดตั้ง dependencies

```
npm install dotenv mysql2 cors
```

```
npm install @types/cors
```

2. สร้างไฟล์ .env เพื่อเก็บตัวแปร environment ดังนี้

```
PORT=4000
```

```
DB_HOST=localhost
```

```
DB_USER=root
```

```
DB_PASSWORD=1234
```

```
DB_NAME=shopdee
```

```
DB_PORT=3306
```

3. แก้ไขไฟล์ src/index.ts

//1. Importing Dependencies

```
.....  
import cors from 'cors';  
import mysql from 'mysql2';  
import dotenv from 'dotenv';  
dotenv.config();
```

//2. Configuration

```
.....  
app.use(express.json()); //ใช้ middleware เพื่อให้สามารถรับข้อมูล JSON ได้  
app.use(express.urlencoded({ extended: true })); //เพื่อให้สามารถรับข้อมูลจาก HTML Form ได้  
app.use(cors()); // อนุญาตให้ API ถูกเรียกจากโดเมนอื่น
```

```
// MySQL Connection  
const db = mysql.createConnection({  
  host: process.env.DB_HOST,  
  user: process.env.DB_USER,  
  password: process.env.DB_PASSWORD,  
  database: process.env.DB_NAME,  
  port: Number(process.env.DB_PORT)  
});  
db.connect();
```

//3. Defining API Endpoints

//สร้างฟังก์ชัน query เพื่อใช้ในการ query ข้อมูลและลดความซ้ำซ้อนในการเขียน

```
function query(sql: string, params: any[] = []):  
  Promise<any> {  
  return new Promise((resolve, reject) => {  
    db.query(sql, params, (err, results) => {  
      if (err) reject(err);  
      else resolve(results);  
    });  
  });  
}  
.....
```

```
// CREATE - เพิ่มลูกค้าใหม่
app.post('/api/customer', async (req, res) => {
  try {
    const { username, password, firstName, lastName } = req.body;
    const sql = `
      INSERT INTO customer (username, password, firstName, lastName)
      VALUES (?, ?, ?, ?)
    `;
    const result = await query(sql, [username, password, firstName, lastName]);
    res.json({
      status: true,
      message: 'Customer created successfully'
    });
  } catch (err) {
    res.status(500).json({
      status: false,
      message: 'Failed to create customer'
    });
  }
});
```

```
// READ - ดึงลูกค้าทั้งหมด
app.get('/api/customer', async (req, res) => {
  try {
    const sql = `SELECT * FROM customer`;
    const customers = await query(sql);
    res.json({
      status: true,
      message: 'Customers fetched successfully',
      data: customers
    });
  } catch (err) {
    res.status(500).json({
      status: false,
      message: 'Failed to fetch customers'
    });
  }
});
```

```
// READ (by ID) - ดึงลูกค้ารายเดียว
app.get('/api/customer/:custID', async (req, res) => {
  try {
    const { custID } = req.params;
    const sql = `SELECT * FROM customer WHERE custID = ?`;
    const results = await query(sql, [custID]);
    if (results.length === 0) {
      res.status(404).json({
        status: false,
        message: 'Customer not found'
      });
    } else {
      res.json({
        status: true,
        message: 'Customer fetched successfully',
        data: results[0]
      });
    }
  } catch (err) {
    res.status(500).json({
      status: false,
      message: 'Failed to fetch customer'
    });
  }
});
```

```
// UPDATE - แก้ไขลูกค้า
app.put('/api/customer/:custID', async (req, res) => {
  try {
    const { custID } = req.params;
    const { username, password, firstName, lastName } = req.body;
    const sql = `
      UPDATE customer
      SET username = ?, password = ?, firstName = ?, lastName = ?
      WHERE custID = ?
    `;
    const result = await query(sql, [username, password, firstName, lastName, custID]);
    if (result.affectedRows === 0) {
      res.status(404).json({
        status: false,
        message: 'Customer not found'
      });
    } else {
      res.json({
        status: true,
        message: 'Customer updated successfully'
      });
    }
  } catch (err) {
    res.status(500).json({
      status: false,
      message: 'Failed to update customer'
    });
  }
});
```

```
// DELETE - ลบลูกค้า
app.delete('/api/customer/:custID', async (req, res) => {
  try {
    const { custID } = req.params;
    const sql = `DELETE FROM customer WHERE custID = ?`;
    const result = await query(sql, [custID]);
    if (result.affectedRows === 0) {
      res.status(404).json({
        status: false,
        message: 'Customer not found'
      });
    } else {
      res.json({
        status: true,
        message: 'Customer deleted successfully'
      });
    }
  } catch (err) {
    res.status(500).json({
      status: false,
      message: 'Failed to delete customer'
    });
  }
});
```

4. ทดสอบ API ด้วย Postman

เพิ่มลูกค้าใหม่ – <http://localhost:4000/api/customer>

The screenshot shows the Postman interface for a POST request to `http://localhost:4000/api/customer`. The request body is set to `x-www-form-urlencoded` with the following data:

	Key	Value	Description
☒	username	somchai	
☒	password	1234	
☒	firstName	สมชาย	
☒	lastName	มันคง	

The response is `200 OK` with a status of `36 ms` and a body size of `324 B`. The response body is JSON:

```
1 {
2   "status": true,
3   "message": "Customer created successfully"
4 }
```

ดึงลูกค้าทั้งหมด – http://localhost:4000/api/customer

The screenshot shows a REST client interface with the following details:

- Method:** GET
- URL:** http://localhost:4000/api/customer
- Status:** 200 OK
- Response Time:** 24 ms
- Response Size:** 630 B

The response body is displayed in JSON format:

```
1 {
2   "status": true,
3   "message": "Customers fetched successfully",
4   "data": [
5     {
6       "custID": 1,
7       "username": "somchai",
8       "password": "1234",
9       "firstName": "สมชาย",
10      "lastName": "มันคง",
11      "address": null,
12      "zipcode": null,
```

ดึงลูกค้ารายเดียว – http://localhost:4000/api/customer/1

The screenshot shows a REST client interface with the following components:

- Method and URL:** GET http://localhost:4000/api/customer/1
- Buttons:** Send (blue), and tabs for Params, Auth, Headers (8), Body, Scripts, Settings, and Cookies.
- Query Params:** A table with columns Key, Value, and Description. It is currently empty.
- Response Status:** 200 OK, 35 ms, 627 B.
- Response Body:** JSON format showing the following data:

```
1 {
2   "status": true,
3   "message": "Customer fetched successfully",
4   "data": {
5     "custID": 1,
6     "username": "somchai",
7     "password": "1234",
8     "firstName": "สมชาย",
9     "lastName": "มันคง",
10    "address": null,
11    "zipcode": null,
12    "mobilePhone": null,
13    "homePhone": null,
```

ลบลูกค้า – http://localhost:4000/api/customer/1

⌵

🔗

🕒

🧩

DELETE ⌵

http://localhost:4000/api/customer/1

Send ⌵

Params

Auth

Headers (9)

Body ●

Scripts

Settings

Cookies

Query Params

	Key	Value	Description	⋮ Bulk Edit
	Key	Value	Description	

Body ⌵ 🕒

200 OK • 28 ms • 324 B • 🌐 ⋮

{{}} JSON ⌵

▶ Preview

🔍 Visualize ⌵

⌵ | ≡ 🔍 | 📄 🔗

```
1  {
2    "status": true,
3    "message": "Customer deleted successfully"
4  }
```

3. สร้าง Register API

1. ติดตั้ง dependencies

```
npm install express-validator  
npm install bcryptjs
```

2. แก้ไขไฟล์ src/index.ts

//1. Importing Dependencies

.....

```
import { Request, Response, NextFunction } from 'express';  
import { body, validationResult } from 'express-validator';  
import bcrypt from 'bcryptjs';
```

//3. Defining API Endpoints

```
.....
// Register
app.post('/api/register', [
  body('username').isString().notEmpty().withMessage('Username is required'),
  body('password').isLength({ min: 4 }).withMessage('Password must be at least 4 characters'),
  body('firstName').isString().notEmpty().withMessage('First name is required'),
  body('lastName').isString().notEmpty().withMessage('Last name is required')
], async (req: Request, res: Response, next: NextFunction) => {
  const errors = validationResult(req);
  if (!errors.isEmpty()) {
    return res.status(400).send({ message: 'Validation errors', errors: errors.array(), status: false });
  }

  const { username, password, firstName, lastName } = req.body;
  try {
    const existingUser = await query("SELECT * FROM customer WHERE username=?", [username]);
    if (existingUser.length > 0) {
      return res.status(409).send({ message: 'Username already exists', status: false });
    }

    const salt = await bcrypt.genSalt(10);
    const password_hash = await bcrypt.hash(password, salt);

    await query('INSERT INTO customer (username, password, firstName, lastName) VALUES (?, ?, ?, ?)',
      [username, password_hash, firstName, lastName]);

    res.send({ message: 'Registration successful', status: true });
  } catch (err) {
    next(err);
  }
});
```

📄

🔗

🕒

🧩

POST

⌵

http://localhost:4000/api/register

Send

⌵

Params

Auth

Headers (9)

Body ●

Scripts

Settings

Cookies

x-www-form-urlencoded

⌵

	Key	Value	Description	⋮ Bulk Edit
☒	username	somsak		
☒	password	1234		
☒	firstName	สมศักดิ์		
☒	lastName	มันคง		
	Key	Value	Description	

Body

⌵

🕒

200 OK

• 100 ms

• 318 B

• 🌐

• ⋮

{ }

JSON

⌵

▶ Preview

🔗 Visualize

⌵

⌵

⌵

🔍

📄

🔗

1 {

2 |

3 | "message": "Registration successful",

4 | "status": true

5 }

4. สร้าง Login API

1. ติดตั้ง dependencies

```
npm install jsonwebtoken
```

```
npm install @types/jsonwebtoken
```

2. แก้ไขไฟล์ .env โดยเพิ่ม SECRET_KEY

```
SECRET_KEY=your_secret_key
```

3. แก้ไขไฟล์ src/index.ts

```
//1. Importing Dependencies
```

```
.....
```

```
import jwt from 'jsonwebtoken';
```

```
//2. Configuration
```

```
.....
```

```
const SECRET_KEY = process.env.SECRET_KEY || 'your_secret_key'; // กำหนด secret key สำหรับ JWT
```

```
.....
```

//3. Defining API Endpoints

```
.....
// Login
app.post('/api/login', [
  body('username').isString().notEmpty().withMessage('Username is required'),
  body('password').isString().notEmpty().withMessage('Password is required')
], async (req: Request, res: Response, next: NextFunction) => {
  const errors = validationResult(req);
  if (!errors.isEmpty()) {
    return res.status(400).send({ message: 'Validation errors', errors: errors.array(), status: false });
  }

  const { username, password } = req.body;
  try {
    const customer = await query("SELECT * FROM customer WHERE username=? AND isActive = 1", [username]);
    if (customer.length === 0) {
      return res.status(401).send({ message: 'Invalid username or password', status: false });
    }

    const isPasswordValid = bcrypt.compareSync(password, customer[0].password);
    if (!isPasswordValid) {
      return res.status(401).send({ message: 'Invalid username or password', status: false });
    }

    const token = jwt.sign({ custID: customer[0].custID, username: customer[0].username }, SECRET_KEY, { expiresIn: '1h' });
    res.send({ custID: customer[0].custID, username, email: customer[0].email, token, message: 'Login successful', status: true });
  } catch (err) {
    next(err);
  }
});
```

POST http://localhost:4000/api/login Send

Params Auth Headers (9) **Body** Scripts Settings Cookies

x-www-form-urlencoded

	Key	Value	Description	...	Bulk Edit
☒	username	somsak			
☒	password	1234			

Body 200 OK • 93 ms • 537 B • ...

{ } JSON Preview Visualize

```
1 {
2   "custID": 2,
3   "username": "somsak",
4   "email": null,
5   "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJqdXN0SUQiOiJpInVzZXJuYW11Ijoic29tc2FrIiwiaWF0IjoxNzUzNDg4MTMyLCJleHAiOiJl3NTM00TE3MzJ9.IAT1obc-VCiGXNW8iQpQGtpOLQil4iNtCA26ZsM2czs",
6   "message": "Login successful",
7   "status": true
8 }
```

นำ token ไปใช้ในหน้า user profile
หรือหน้าอื่น ๆ ที่ต้องการเข้าถึง API

6. สร้าง Profile API

//3. Defining API Endpoints

```
// Interface for JWT payload
interface JwtPayload {
  custID: number;
  username: string;
  positionID?: number;
}

// Function to validate JWT token
function validateToken(req: Request, res: Response, next: NextFunction) {
  const token = req.headers['authorization']?.split(' ')[1];
  if (!token) return res.status(401).json({ message: 'Access denied' });

  try {
    const decoded = jwt.verify(token, SECRET_KEY) as JwtPayload;
    (req as any).user = decoded;
    next();
  } catch (err) {
    res.status(403).json({ message: 'Invalid token' });
  }
}

.....
```

```

.....
// User Profile
app.get('/api/profile/:id', validateToken, async (req: Request, res: Response, next: NextFunction) => {
  const custID = req.params.id;
  const user = (req as any).user as JwtPayload;

  if (Number(custID) !== user.custID) {
    return res.status(403).send({ message: 'Access denied', status: false });
  }

  try {
    const customer = await query(`
      SELECT custID, username, firstName, lastName, address, mobilePhone, gender, email, imageFile,
        IF(birthdate IS NOT NULL, DATE_FORMAT(birthdate, '%Y-%m-%d'), NULL) AS birthdate
      FROM customer
      WHERE custID = ? AND isActive = 1`, [custID]);

    if (customer.length === 0) {
      return res.status(404).send({ message: 'Customer not found', status: false });
    }

    res.send({ ...customer[0], message: 'Success', status: true });
  } catch (err) {
    next(err);
  }
});

```

GET ▼ http://localhost:4000/api/profile/2 Send ▼

Params **Auth** ● Headers (8) Body Scripts Settings Cookies

Auth Type

Bearer Token ▼ Token

ใช้ token ที่ได้จากการ login

เลือก Bearer Token

200 OK • 6 ms • 500 B • ⋮

{} JSON ▼ ▶ Preview 🔗 Visualize ▼

```
1 {
2   "custID": 2,
3   "username": "somsak",
4   "firstName": "สมศักดิ์",
5   "lastName": "มันคง",
6   "address": null,
7   "mobilePhone": null,
8   "gender": null,
9   "email": null,
10  "imageFile": null,
11  "birthdate": null,
12  "message": "Success",
13  "status": true
14 }
```

- **API สำหรับแสดงภาพโปรไฟล์**

//1. Importing Dependencies

```
.....  
import path from 'path';
```

//3. Defining API Endpoints

```
.....  
//Image Profile API  
app.get('/api/customer/image/:filename', (req: Request, res: Response) => {  
  const filepath = path.join(__dirname, '../assets', 'customer', req.params.filename);  
  res.sendFile(filepath);  
});
```

Note : เก็บภาพไว้ในโฟลเดอร์ assets/customer/

• API สำหรับอัปโหลดภาพโปรไฟล์

//1. Importing Dependencies

```
.....  
import fs from "fs";  
import fileUpload, { UploadedFile } from "express-fileupload";
```

//2. Configuration

```
.....  
app.use(fileUpload());
```

//3. Defining API Endpoints

```
.....  
// Uploading Image Profile API  
app.put(  
  "/api/customer/image/upload/:id", validateToken,  
  async (req: Request, res: Response) => {  
  
    const custID = req.params.id;  
  
    //กำหนดให้ user รับค่าจาก req (token) และมีชนิดเป็น JwtPayload  
    const user = (req as any).user as JwtPayload;
```

//ติดตั้ง Library

```
npm install express-fileupload  
npm install -D @types/express-fileupload
```

```
if (Number(custID) !== user.custID) {
  return res.status(403).send({ message: 'Access denied', status: false });
}

try {
  // save file into folder
  let fileName = "";
  if (req.files?.imageFile) {
    const imageFile = req.files.imageFile as UploadedFile;

    const nameParts = imageFile.name.split(".");
    const ext = nameParts.pop();
    const baseName = nameParts.join(".");
    fileName = `${baseName}${Date.now()}.${ext}`;

    const imagePath = path.join(__dirname, "../assets/customer", fileName);

    fs.writeFile(imagePath, imageFile.data, (err) => {
      if (err) throw err;
    });
  } else {
    return res.send({
      message: "กรุณาเลือกไฟล์รูปภาพ",
      status: false,
    });
  }
}
```

```
const sql = `UPDATE customer SET imageFile = ? WHERE custID = ?`;
const result = await query(sql, [fileName, custID]);
if (result.affectedRows === 0) {
    res.status(404).json({
        status: false,
        message: 'Customer not found'
    });
} else {
    res.json({
        status: true,
        message: 'Profile picture updated successfully'
    });
}
} catch (err) {
    res.status(500).json({
        status: false,
        message: 'Failed to upload image'
    });
}
}
);
```

5. แบบฝึกหัด

1. จงอธิบายว่า Express.js คืออะไร และเหตุใดจึงได้รับความนิยมในการสร้าง API ด้วย Node.js มากกว่าการใช้โมดูล http มาตรฐาน
2. Middleware ใน Express.js คืออะไร? จงอธิบายหลักการทำงานและยกตัวอย่างการนำไปใช้งานจริงมา 2 ตัวอย่าง
3. HTTP Request Method คืออะไร และมีหน้าที่อย่างไรในการสื่อสารระหว่าง Client และ Server พร้อมอธิบายความหมายของกลุ่ม Status Code ต่อไปนี้
 - GET
 - POST
 - PUT
 - PATCH
 - DELETE
 - HEAD
 - OPTIONS

4. HTTP Status Code คืออะไร และมีหน้าที่อย่างไรในการสื่อสารระหว่าง Client และ Server พร้อมอธิบายความหมายของกลุ่ม Status Code ต่อไปนี้

- 1xx
- 2xx
- 3xx
- 4xx
- 5xx

5. จงอธิบายความหมายและยกตัวอย่างสถานการณ์การใช้งานของ Status Code ต่อไปนี้

- 200 OK
- 201 Created
- 400 Bad Request
- 401 Unauthorized
- 403 Forbidden
- 404 Not Found
- 500 Internal Server Error

6. จงอธิบายความแตกต่างและการใช้งานของ req.body, req.params, และ req.query
7. จงอธิบายขั้นตอนการทำงานของ Express.js เมื่อผู้ใช้ส่ง HTTP Request มายังเซิร์ฟเวอร์
ให้อธิบายลำดับการทำงานตั้งแต่

- Client
- Request
- Middleware
- Route Handler
- Response

พร้อมวาดแผนภาพประกอบ

8. จงอธิบายความแตกต่างของ API ในแต่ละประเภทระหว่าง REST, GraphQL, และ gRPC
9. จงอธิบายความแตกต่างระหว่าง REST API, Webhook, และ Websocket

10. จงสร้าง API เกี่ยวกับการคำนวณ ดังต่อไปนี้

- คำนวณการบวก
- คำนวณการลบ
- คำนวณเส้นรอบวง
- คำนวณพื้นที่วงกลม

11. พัฒนาส่วน Backend (API) ของระบบซื้อขายออนไลน์ โดยเลือกพัฒนา อย่างน้อย 5 features ต่อไปนี้

• ส่วนของลูกค้า

- | | |
|---|-------------------------------|
| 1. หน้าหลัก (เฉพาะส่วนโครงสร้างและเมนู) | 9. เพิ่มสินค้าเข้าตะกร้า |
| 2. สมัครสมาชิก | 10. แสดงข้อมูลในตะกร้าสินค้า |
| 3. เข้าสู่ระบบ | 11. แก้ไขข้อมูลในตะกร้าสินค้า |
| 4. แสดงโปรไฟล์/รูปโปรไฟล์ | 12. ยืนยันการสั่งซื้อ |
| 5. แก้ไขโปรไฟล์/รูปโปรไฟล์ | 13. ยืนยันชำระเงิน |
| 6. ออกจากระบบ | 14. แสดงประวัติการสั่งซื้อ |
| 7. หน้าแรก (รายการสินค้า) | |
| 8. รายละเอียดสินค้า | |

• ส่วนของพนักงาน

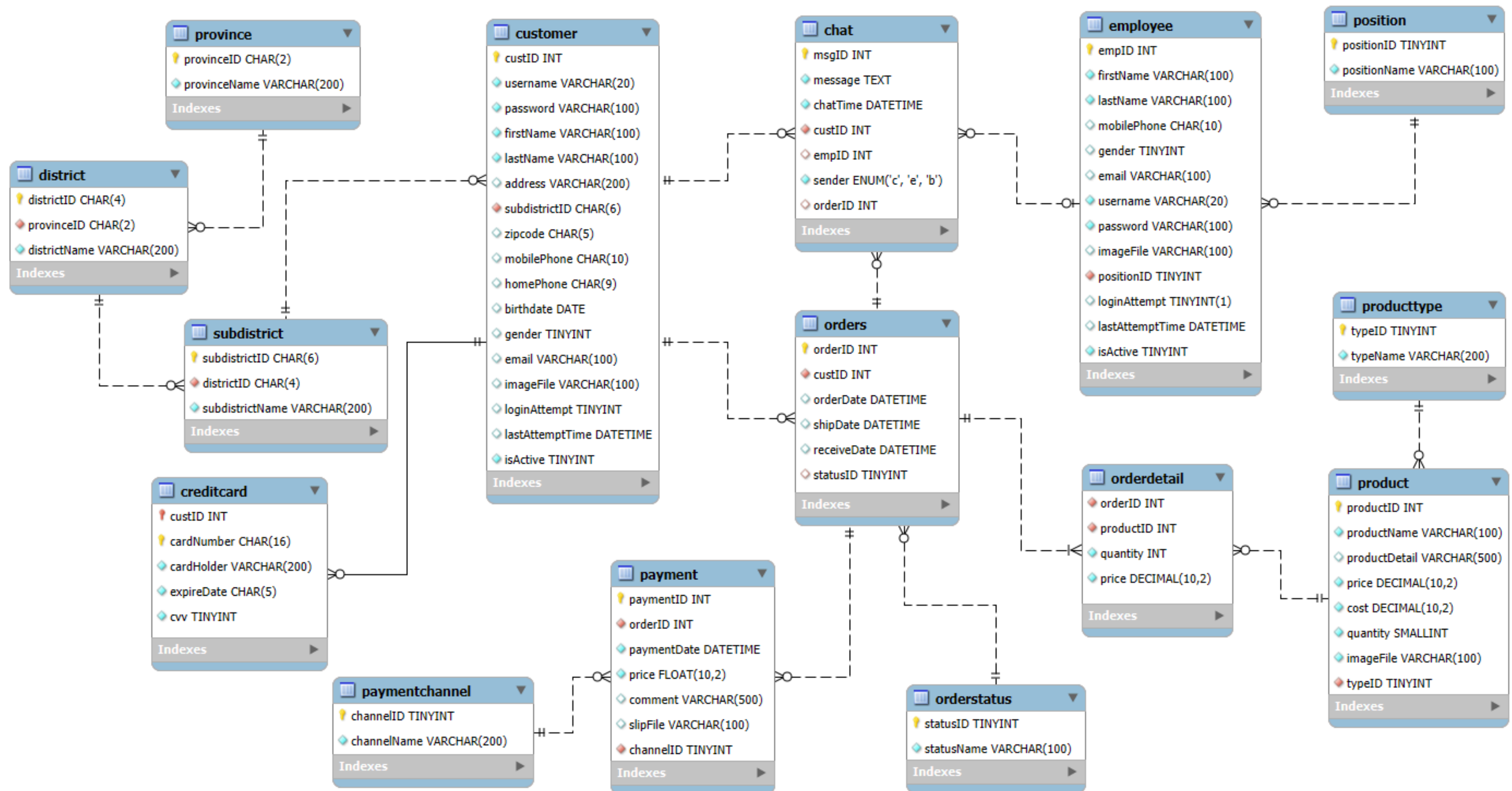
1. เข้าสู่ระบบ
2. โปรไฟล์พนักงาน/รูปโปรไฟล์
3. จัดการข้อมูลลูกค้า
4. จัดการข้อมูลสินค้า
5. จัดการข้อมูลประเภทสินค้า
6. จัดการข้อมูลจังหวัด
7. จัดการข้อมูลอำเภอ
8. จัดการข้อมูลตำบล
9. ตรวจสอบและอนุมัติสลิปเงิน
10. รายงานการ ซื้อขาย (รายวัน)

• ส่วนของผู้บริหาร มีทุก features เหมือนส่วนของพนักงาน ยกเว้นรายการต่อไปนี้

1. จัดการข้อมูลพนักงาน
2. รายงานการ ซื้อขาย (รายวัน/เดือน/ไตรมาส/ปี)

```
project-name/  
├── frontend/  
│   ├── nextjs/  
│   └── flutter/  
└── backend/  
    ├── express/  
    │   └── assets/  
    │       ├── customer/  
    │       ├── employee/  
    │       ├── product/  
    │       └── payment/  
    ├── flask/  
    └── mysql/  
        └── schema.sql
```

ตัวอย่างโครงสร้างไฟล์เตอร์ของ Project



ตัวอย่าง ER-Diagram ของระบบซื้อขายออนไลน์

THANK YOU

FOR YOUR ATTENTION



Contact Me:

 ห้อง 970

 pichai_jo@rmutto.ac.th

 busit.rmutto.ac.th

